

```

/* =====
script.js
ใช้ร่วมกันทุกหน้า: product.html / order.html / admin.html
ตรวจจับหน้าปัจจุบันจาก element ที่มีอยู่ในหน้านั้น
=====
*/
const APPS_SCRIPT_URL = 'https://script.google.com/macros/s/
AKfycbwZwEVuifAC_I0l58ruGy3bRGE8m1wEksEezNsvn2xokQ7JvXjk4RPBam0
Xltb4ZR2Hg/exec';
const SHEET_CSV_URL = 'https://docs.google.com/spreadsheets/d/
e/2PACX-1vQyvPmW0hRRenf8d2pXhCYC6Y8omwEUv7mR98N30Nj7-
E2ftQPMv0tC40eZFsL5QzrVM3T2xI0EfUIId/pub?
gid=0&single=true&output=csv';
const ALL_MOOD_LABEL = 'ทั้งหมด';

document.addEventListener('DOMContentLoaded', () => {
  if (document.querySelector('#product-list')) {
    initProductPage();
  }
  if (document.querySelector('#orderForm')) {
    initOrderPage();
  }
  if (document.querySelector('#ordersTable tbody')) {
    initAdminPage();
  }
});

/* =====
1) product.html
=====
*/
function initProductPage() {
  const filterBar = document.querySelector('#filter-bar');
  const productList = document.querySelector('#product-list');

  const params = new URLSearchParams(window.location.search);
  let currentMood = params.get('mood') || ALL_MOOD_LABEL;

  let allProducts = [];

  fetch('products.json')
    .then(res => res.json())
    .then(data => {
      allProducts = Array.isArray(data) ? data : [];
    });
}

```

```

// สร้างรายการ mood แบบไดนามิกจากข้อมูลที่ดาวน์โหลดมาจริง
// แทนการใช้ array ที่ hardcode ไว้
const moods = [ALL_MOOD_LABEL, ...new Set(
  allProducts
    .map(p => p.mood)
    .filter(m => typeof m === 'string' && m.trim() !==
''))
  )];

// normalize: ถ้า mood ใน URL ไม่ตรงกับ list ที่รู้จัก ให้ fallback
เป็น ALL_MOOD_LABEL
const matched = moods.find(m => m.toLowerCase() ===
currentMood.toLowerCase());
currentMood = matched || ALL_MOOD_LABEL;

renderFilterBar(filterBar, moods, currentMood, (mood) =>
{
  currentMood = mood;
  renderProductList(productList, allProducts,
currentMood);
  highlightActiveFilter(filterBar, currentMood);
});

renderProductList(productList, allProducts, currentMood);
highlightActiveFilter(filterBar, currentMood);
})
.catch(error => {
  console.error(error);
  productList.innerHTML = '<p>ไม่สามารถโหลดสินค้าได้ กรุณาลองใหม่อีก
ครั้ง</p>';
});
}

function renderFilterBar(filterBar, moods, currentMood,
onSelect) {
  filterBar.innerHTML = '';
  moods.forEach(mood => {
    const btn = document.createElement('button');
    btn.type = 'button';
    btn.textContent = mood;
    btn.dataset.mood = mood;
    btn.addEventListener('click', () => onSelect(mood));
    filterBar.appendChild(btn);
  });
}

```

```
function highlightActiveFilter(filterBar, currentMood) {
  const buttons = filterBar.querySelectorAll('button');
  buttons.forEach(btn => {
    if (btn.dataset.mood === currentMood) {
      btn.classList.add('active');
    } else {
      btn.classList.remove('active');
    }
  });
}
```

```
function renderProductList(productList, products, mood) {
  const filtered = (mood === ALL_MOOD_LABEL)
    ? products
    : products.filter(p => p.mood === mood);
```

```
  productList.innerHTML = '';
```

```
  if (filtered.length === 0) {
    productList.innerHTML = '<p>ไม่พบสินค้าในหมวดนี้</p>';
    return;
  }
```

```
  filtered.forEach(product => {
    const card = document.createElement('div');
    card.className = 'product-card';
```

```
    const img = document.createElement('img');
    img.src = product.image || '';
    img.alt = product.name || '';
    card.appendChild(img);
```

```
    const name = document.createElement('h3');
    name.textContent = product.name || '';
    card.appendChild(name);
```

```
    const size = document.createElement('p');
    size.className = 'product-size';
    size.textContent = product.size || '';
    card.appendChild(size);
```

```
    const price = document.createElement('p');
    price.className = 'product-price';
    price.textContent = (product.price !== undefined &&
```

```

product.price !== null)
    ? `${product.price} บาท`
    : '';
card.appendChild(price);

const orderBtn = document.createElement('a');
orderBtn.className = 'order-btn';
orderBtn.textContent = 'สั่งซื้อ';
const orderParams = new URLSearchParams();
orderParams.set('item', product.name || '');
orderParams.set('price', product.price !== undefined &&
product.price !== null ? product.price : '');
orderBtn.href = `order.html?${orderParams.toString()}`;
card.appendChild(orderBtn);

productList.appendChild(card);
});
}

/* =====
2) order.html
=====
*/
function initOrderPage() {
    const form = document.querySelector('#orderForm');
    const itemsField = document.querySelector('#items');
    const totalField = document.querySelector('#total');
    const customerNameField =
document.querySelector('#customerName');
    const contactField = document.querySelector('#contact');
    const noteField = document.querySelector('#note');

    const params = new URLSearchParams(window.location.search);
    const item = params.get('item');
    const price = params.get('price');

    if (itemsField && item !== null) {
        itemsField.value = item;
    }
    if (totalField && price !== null) {
        totalField.value = price;
    }

    form.addEventListener('submit', (e) => {
        e.preventDefault();
    });
}

```

```

const payload = {
  customerName: customerNameField ?
customerNameField.value : '',
  contact: contactField ? contactField.value : '',
  items: itemsField ? itemsField.value : '',
  total: totalField ? totalField.value : '',
  note: noteField ? noteField.value : ''
};

fetch(APPS_SCRIPT_URL, {
  method: 'POST',
  body: JSON.stringify(payload)
})
  .then(() => {
    window.location.href = 'thankyou.html';
  })
  .catch(error => {
    console.error(error);
    alert('เกิดข้อผิดพลาด กรุณาลองใหม่อีกครั้ง');
  });
});
}

```

```

/* =====
3) admin.html
=====
*/
function initAdminPage() {
  const tbody = document.querySelector('#ordersTable tbody');

  fetch(SHEET_CSV_URL)
    .then(res => res.text())
    .then(csvText => {
      const rows = parseCSV(csvText);

      if (rows.length === 0) {
        tbody.innerHTML = '<tr><td colspan="6">ไม่พบข้อมูล</td></tr>';
        return;
      }

      // แถวแรกสมมติเป็น header -> ตัดออก
      const dataRows = rows.slice(1).filter(r => r.some(cell =>
cell.trim() !== ''));

```

```

// เรียงลำดับขึ้นก่อน โดยอิงคอลัมน์แรก (วันเวลา)
dataRows.sort((a, b) => {
    const dateA = new Date(a[0]);
    const dateB = new Date(b[0]);
    const timeA = isNaN(dateA.getTime()) ? 0 :
dateA.getTime();
    const timeB = isNaN(dateB.getTime()) ? 0 :
dateB.getTime();
    return timeB - timeA;
});

tbody.innerHTML = '';
dataRows.forEach(row => {
    const tr = document.createElement('tr');
    for (let i = 0; i < 6; i++) {
        const td = document.createElement('td');
        td.textContent = row[i] !== undefined ? row[i] : '';
        tr.appendChild(td);
    }
    tbody.appendChild(tr);
});
})
.catch(error => {
    console.error(error);
    tbody.innerHTML = '<tr><td colspan="6">ไม่สามารถโหลดข้อมูลได้
กรุณาลองใหม่อีกครั้ง</td></tr>';
});
}

/* CSV parser แบบง่าย เขียนเอง ไม่ใช่ library ภายนอก
รองรับ: field คั่นด้วย comma, field ที่ครอบด้วย double quote,
double quote ซ้อนภายใน field (") และ newline ภายใน quoted
field */
function parseCSV(text) {
    const rows = [];
    let row = [];
    let field = '';
    let inQuotes = false;

    // normalize line endings
    const str = text.replace(/\r\n/g, '\n').replace(/\r/g, '\n');

    for (let i = 0; i < str.length; i++) {
        const char = str[i];

```

```

const nextChar = str[i + 1];

if (inQuotes) {
  if (char === '"' && nextChar === '"') {
    field += '"';
    i++;
  } else if (char === '"') {
    inQuotes = false;
  } else {
    field += char;
  }
} else {
  if (char === '"') {
    inQuotes = true;
  } else if (char === ',') {
    row.push(field);
    field = '';
  } else if (char === '\n') {
    row.push(field);
    field = '';
    rows.push(row);
    row = [];
  } else {
    field += char;
  }
}
}

// เก็บ field/row สุดท้ายที่เหลือ
if (field.length > 0 || row.length > 0) {
  row.push(field);
  rows.push(row);
}

return rows;
}

```